САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 1

Выполнил:

Лорс Алина, 474321

Группа J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Я выбрала идею сайта: Сервис для управления личными финансами и бюджетом

Задача: создать проект(в моем случае название это Tarelka) управления личными финансами,в котором выполняется тз. Продемонстрировать фронтенд-разработку рабочего интерфейса и протестировать все функции.

ТЗ ЗАДАНИЯ

- Вход
- Регистрация
- Личный кабинет пользователя (счета, транзакции, бюджеты)
- Поиск и фильтрация транзакций по категории, сумме и дате
- Страница отчёта (графики расходов, категории, прогнозы)
- Интеграция с платежными аккаунтами (импорт транзакций, настройка правил)

ТЗ из задания выполнено

Требование задания		Где реализовано	Как это работает
Вход		index.html + js/auth.js	Форма входа проверяет email и пароль, ищет пользователя в localStorage и переводит в кабинет.
Регистрация		register.html + js/auth.js + js/storage.js	Создаётся новый пользователь, валидируются поля, после чего генерируются стартовые данные.
Личный кабинет		dashboard.html + js/dashboard.js	Показываются счета, последние операции, бюджеты и финансовые показатели.
Поиск и фильтрация транзакций		transactions.html + js/transactions.js + js/storage.js	Поддерживается фильтрация по категории, сумме, дате, типу, статусу и поиск по тексту.
Страница отчёта		reports.html + js/reports.js	Строятся графики расходов, категорий, доходов и прогресса бюджетов.
Интеграция с платёжными аккаунтами		integrations.html + js/integrations.js	Есть подключение сервисов, правила

			импорта и загрузка транзакций из CSV.

2 требование

-это иметь в проекте HTML, CSS, JS, Bootstrap:

При регистрации пользователь получает демо-данные автоматически. Это сделано для удобства демонстрации, чтобы преподаватель не видел пустой интерфейс.
-Bootstrap я использовал для адаптивной сетки, готовых форм, таблиц, модальных окон, toast-уведомлений и единообразного современного интерфейса.

- Внутри storage.js реализована основная бизнес-логика: расчёт балансов, фильтрация транзакций, формирование отчётов и обработка импорта.

-Интеграции в этом проекте являются имитацией реального подключения: пользователь может настроить сервис, импортировать CSV и правила обработки, что соответствует прототипу интеграции.

Общая схема работы:

1. Пользователь заходит на страницу → requireAuth() проверяет вход
2. ui.js отрисовывает шапку, меню и подвал
3. Специфичный JS-файл (dashboard/transactions/reports/integrations) запрашивает данные через storage.js
4. storage.js читает из localStorage и возвращает подготовленные данные
5. JS-файл рендерит интерфейс, используя ui.js для форматирования
6. Пользователь что-то меняет → обновление идёт через storage.js → localStorage обновляется → интерфейс перерисовывается

Полный пользовательский сценарий

1. Первый вход:

Пользователь открывает index.html. Это точка входа. Если он уже залогинен, система сразу перекидывает его в личный кабинет. Но если пользователь новый, он видит форму входа и кнопку «Зарегистрироваться».Нажимает на неё и попадает на register.html.

2. Регистрация: создание аккаунта и демо-среды

На странице регистрации пользователь заполняет поля:

имя и фамилия

email

пароль (показывается индикатор силы пароля)

подтверждение пароля

базовая валюта

финансовая цель (накопление на отпуск, покупка авто и т.п.)

После отправки формы auth.js проверяет корректность данных. Если всё хорошо, вызывается storage.registerUser(), который:создаёт нового пользователя

вызывает data.createDemoSeed() для генерации стартовых данных: счета (наличные, карта, вклад), транзакции за последние недели, бюджеты по категориям, несколько правил импорта и тестовую интеграцию.Пользователь не видит пустого аккаунта ,он сразу попадает в наполненную систему.

3. *Личный кабинет: центр управления*

После регистрации пользователь оказывается в dashboard.html. Здесь он видит:

<u>Верхние метрики:</u>

- общий баланс по всем счетам
- доходы за месяц
- расходы за месяц
- прогресс по финансовой цели

карточки с названием счёта, текущим балансом и иконкой сразу видно, сколько денег в наличных, сколько на карте

Последние транзакции: писок из 5–10 свежих операций с датой, категорией, суммой и иконкой. можно быстро понять, куда уходили деньги в последние дни

Бюджеты: progress-бары показывают, сколько потрачено из запланированного лимита по каждой категории.если осталось мало, прогресс-бар подсвечивается красным

Кнопки быстрых действий: добавить доход добавить расход перевести между счетами

Пользователь может сразу добавить новую транзакцию открывается модальное окно, заполняет сумму, категорию, счёт, и данные сохраняются через storage.js. Балансы и списки обновляются без перезагрузки страницы

4. Транзакции: поиск, фильтрация, управление

Если пользователь хочет детально разобраться со всеми операциями, он переходит в раздел transactions.html.

- фильтры по категории, счёту, типу (доход/расход), статусу (выполнено/запланировано), диапазону сумм и дат
- строка поиска : ищет по описанию, категории и счёту
- переключение режимов : можно смотреть транзакции в виде компактной таблицы или визуальных карточек с иконками
- пагинация, чтобы не загружать все транзакции сразу

При клике на любую транзакцию открывается модальное окно с подробностями: сумма, дата, категория, счёт, описание. Там же можно изменить сумму, категорию, дату. И удалить если операция была ошибочной или дублируется. Если удаляемая транзакция была частью перевода (привязана к transfer_id), система удаляет обе связанные записи исходящую и входящую, чтобы балансы счетов оставались корректными дада вот так то

5. Отчёты: аналитика и визуализация

Пользователь переходит в reports.html, чтобы увидеть картину своих финансов целиком.

Сверху карточки с ключевыми метриками:доходы за выбранный период; расходы за выбранный период;средний чек;доминирующая категория расходов

С помощью селектора можно переключать период: неделя, месяц, квартал, год. При смене периода все метрики и графики пересчитываются автоматически,магия вне хогвартса ю нов.

Графики, которые я построила через Chart.js:

расходы по категориям(круговая диаграмма, сразу видно, куда утекает бюджет)

1. динамика трат (линейный график, показывает, как менялись расходы день за днём)
2. сравнение доходов и расходов (столбчатая диаграмма, наглядно видно, в плюсе пользователь или в минусе)
3. прогресс по бюджетам (горизонтальная диаграмма, показывает, насколько исчерпаны лимиты по каждой категори)

6. Интеграции: автоматизация сбора данных

Раздел integrations.html я вела для тех кто не хочет вводить транзакции вручную(мало ли)

Пользователь видит карточки сервисов т банк и спб банк

Для каждого можно включить интеграцию (тумблером), ввести токен доступа (из личного кабинета банка или платформы),задать дату, с которой начинать синхронизацию

Импорт CSV:
можно загрузить выписку из любого банка в формате CSV. Система:парсит файл через parseCsv() затем применяет созданные правила и еще умеет добавляет транзакции через importTransactions()

Для демонстрации есть кнопка «Импортировать тестовый CSV» она загружается sampleImportCsv из data.js и пользователь видит, как работает процесс, не подготавливая свой файл. История импортов: отображается список всех загрузок: когда импортировался файл, сколько транзакций добавилось, были ли ошибки.

8. Постоянство данных

В течение всего сценария все изменения:

(добавление транзакций.редактирование и удаление.создание правил.импорт CSV)

мгновенно сохраняются в localStorage через storage.js. Никакой ручной синхронизации или повторного ввода,ура

По самому коду:

описание HTML-файлов

index.html страница служит точкой входа в приложение. форма с полями "Email" и "Пароль".

После отправки дёргается логин из auth.js, который чекает пользователя в localStorage. Если пользователь уже залогинен, его сразу перекидывает на dashboard.html. Проверка делается ещё до отрисовки формы. Я вынесла файл отдельно от основного интерфейса, так сложнее случайно сломать.

Bootstrap (CSS + JS)

Bootstrap Icons

Плюс подключил: auth.css -стили конкретно под форму входа. common.css — общие стили для всего приложения

register.html Страница регистрации нового пользователя.

Здесь собираю все данные для создания аккаунта.

-Сделала индикатор силы пароля. Плюс добавила валидацию полей в реальном времени, чтобы ошибки подсвечивались сразу, а не после отправки.

-выбор базовой валюты. финансовая цель (чтобы сразу понимать, под что настраивать демо-данные)

после успешной регистрации я создаю нового пользователя в localStorage и сразу генерирую демо-среду заполняю тестовыми транзакциями, категориями и стартовым балансом, чтобы человек мог сразу увидеть, как работает приложение, не вводя ничего вручную. После сохранения данных делаю переход в личный кабинет (dashboard.html).

dashboard.html Главный экран после входа это личный кабинет пользователя.

Это центральная страница всего проекта. Здесь показываю приветствие, краткие метрики (общий баланс, доходы за месяц, расходы за месяц), карточки счетов, последние транзакции и бюджеты по категориям.

Быстрые действия вынесла в кнопки:

- добавить доход
- добавить расход
- перевести между счетами

Для этих действий сделала несколько Bootstrap-модальных окон:

- добавление транзакции (выбор счёта, категории, даты)
- создание нового счёта
- изменение бюджета по категориям
- перевод между счетами

Про организацию кода: часть интерфейса (sidebar с навигацией, topbar с профилем пользователя, footer) вставляется динамически через ui.js. чтобы не было дублирования

transactions.html Страница работы со всеми транзакциями.

Здесь сделала полноценный инструмент для управления всеми финансовыми операциями пользователя.
Фильтры вынесла в отдельную панель. Можно отсеять транзакции по:

- категории (еда, транспорт, развлечения и т.д.) ; диапазону дат
- счёту (наличные, карта, вклад); типу (доход/расход)
- статусу (выполнено, запланировано, ожидается)4 сумме (диапазон от и до)

Добавила строку поиска она ищет по описанию, категории и счёту одновременно.
Режимы отображения:Таблица. карточки
Для навигации по списку реализовала пагинацию -загружаю данные порциями, чтобы страница не тормозила, даже если у пользователя сотни транзакций.
с транзакцией при клике открывается модальное окно с подробностями операции, там же доступны кнопки редактирования и удаления. Всё работает через JavaScript.

reports.html Страница аналитики и отчётов.

Собрала всю визуальную аналитику, чтобы пользователь мог видеть где тратит, сколько зарабатывает, какие категории съедают больше всего денег.

Ключевые метрики вынесла в карточки сверху:

- доходы за выбранный период
- расходы за выбранный период
- средний чек (общая сумма расходов / количество транзакций)
- доминирующая категория (та, на которую ушло больше всего средств)

Графики сделала с помощью Chart.js

Переключение периода анализа добавила. Доступны варианты: Неделя.месяц.квартал.год

integrations.html Страница интеграций и импорта транзакций.

Здесь можно подключить банки и платёжные сервисы, настроить автоматическую загрузку данных или импортировать их из файла.

Внешние сервисы — вывела список доступных интеграций

Для каждого сервиса можно:включить или отключить интеграцию (тумблером). настроить токен доступа (беру из личного кабинета банка или платформы)ю указать дату, с которой начинать синхронизацию чтобы не тянуть транзакции за всё время

Импорт из файла сделала через CSV.я паршу его и добавляю данные в приложение. Там же есть история импортов: видно, когда загружался файл, сколько транзакций добавилось, были ли ошибки.

JS файлы:

js/data.jsФайл инициализации приложения.

содержит словари приложения, список доступных валют, категории, типы счетов и каталог интеграций. Также здесь формируется стартовый набор демонстрационных данных для нового пользователя. В текущей версии демо-среда упрощена: используется небольшой набор стартовых записей, включающий 3–4 транзакции с датами, начиная с 10.03.2026.

Главная функция createDemoSeed().

Она создаёт стартовый набор данных для нового пользователя: счета, бюджеты, транзакции, интеграции, правила импорта и историю импортов. после регистрации человек не видит пустой экран, а попадает в уже наполненное приложение.

Функция relativeDate()

помогает генерировать даты относительно текущего дня. Я использую её, чтобы демо-данные всегда выглядели актуальными .вчерашние операции, расходы за прошлую неделю, переводы, которые только что прошли.

Также здесь лежит sampleImportCsv

тестовый CSV-файл для демонстрации импорта. Пользователь может нажать кнопку и посмотреть, как работает импорт, не подготавливая свой файл.

js/storage.js

Главный слой работы с данными и localStorage. работа с данными, их хранение, изменение и расчёты происходят здесь. Интерфейс только обращается к этому модулю и отрисовывает то, что он вернул.

в localStorage: пользователи, текущий пользователь, счета, транзакции, бюджеты, интеграции, правила импорта, история импортов. Каждый тип данных лежит под своим ключом.Сделала базовые функции read() и write() читают и записывают JSON

Авторизация и защита:

- registerUser() регистрация нового пользователя
- login() вход в систему
- requireAuth() защита внутренних страниц от открытия без входа (если пользователь не залогинен, кидаю на страницу логина)

CRUD-операции:

- addAccount(), addTransaction(), updateTransaction(), deleteTransaction(), updateBudget() для управления основными сущностями
- addTransfer() создаёт сразу две связанные записи (исходящий перевод с одного счёта и входящий на другой), чтобы балансы всегда сходились

Импорт и правила:

- importTransactions() загружает данные из CSV
- applyRulesToTransaction() проверяет транзакцию против настроенных правил и автоматически назначает категорию

Аналитика:

- getDashboardMetrics() сводные показатели для главной страницы
- filterTransactions() фильтрация транзакций по всем параметрам
- getReportData() агрегированные данные для графиков и отчётов

js/auth.js Логика страниц входа и регистрации.

Так проще поддерживать код, легче тестировать, и если что-то нужно поменять в проверке пароля или индикаторе силы, я правлю только здесь

Проверки и валидация:

--setError() выводит ошибки и подсвечивает некорректные поля

--validateEmail() проверяет формат email

UX-фишки:

--togglePassword() кнопка показа/скрытия пароля, чтобы пользователь мог проверить, что ввёл

--updatePasswordStrength() индикатор силы пароля: слабый, средний, надёжный

Основные функции:

--initLogin() обрабатывает вход в систему и сохраняет опцию "запомнить email"

--initRegister() проверяет форму регистрации, сверяет пароли и вызывает BudgetFlowStorage.registerUser()

js/ui.jsОбщий UI-модуль для повторно используемых элементов.

Всё, что может пригодиться на нескольких страницах, я складывала сюда.Компоненты навигации: функции, которые вставляют sidebar, topbar и footer на внутренние страницы. Меню вызываю одну функцию

Форматирование:

--formatCurrency() приводит числа к единообразному формату валюты (с учётом знака и количества знаков после запятой)

--formatDate() единый формат отображения дат для всего приложения

Вспомогательные функции:

--fillSelect()программно заполняет выпадающие списки (категории, счета, валюты) из массивов данных

--showToast() уведомления через Bootstrap Toast: успех, ошибка, предупреждение

s--tatusMarkup() и typeLabel() визуальное представление статусов и типов операций (иконки + цветовые индикаторы)

js/dashboard.js Логика личного кабинета

главной страницы после входа.Метрики и отображения получаю сводные показатели через getDashboardMetrics(). Карточки с общим балансом, доходами за месяц, расходами за месяц. Прогресс по бюджетам с помощью progress-баров

Счета: список всех счетов с актуальным балансом (рассчитывается на основе транзакций

Последние операции: 5–10 последних транзакций и бюджеты с визуализацией, сколько осталось до лимита

Модальные окна: добавляю транзакцию (доход/расход). новый счёт

перевожу между счетами. изменяю бюджет

Каждое окно отдельная форма с валидацией. После сохранения данные обновляются и интерфейс перерисовывается

js/transactions.jsЛогика страницы списка транзакций.

- Хранит состояние страницы: текущие фильтры, режим отображения, номер страницы и размер пагинации.
- Создаёт фильтры и синхронизирует форму фильтрации с внутренним состоянием.
- Получает от storage.js уже отфильтрованные данные и рендерит их в таблицу или карточки.
- Реализует переключение вида, пагинацию, детальный просмотр и редактирование транзакций.
- Позволяет удалять запись; если запись является частью перевода, удаляются обе связанные операции.

js/reports.jsЛогика страницы отчётов и графиков.

Он показывает, что приложение умеет не только хранить операции, но и интерпретировать их визуально.

• Запрашивает агрегированные данные из storage.js для выбранного периода.
• Показывает четыре аналитические карточки с краткими выводами.
• Строит графики через Chart.js: круговой по категориям, линейный по динамике, столбчатый по доходам/расходам и горизонтальный по бюджетам.
• Уничтожает старые экземпляры графиков перед перерисовкой, чтобы избежать наложения canvas и утечки памяти.

js/integrations.jsЛогика страницы интеграций, правил и импорта.

Файл нужен, чтобы показать сценарий интеграции с внешними платёжными системами даже в frontend-прототипе без реального банковского API.

- Отрисовывает карточки подключаемых сервисов и их текущий статус.
- Управляет подключением/отключением интеграций и настройкой синхронизации.
- Создаёт и удаляет правила автоматической обработки импортируемых данных.
- Реализует parseCsv() для чтения CSV и importTransactions() через storage.js.
- Позволяет импортировать как выбранный пользователем CSV, так и тестовый sampleImportCsv.
- Показывает историю импортов.

Подробное описание CSS-файлов

css/app.css

Базовые глобальные стили приложения: общая палитра, layout, sidebar, topbar, footer, общие карточки и системные компоненты.

css/auth.css

Стили страниц входа и регистрации: центрирование формы, оформление карточки авторизации, индикатор силы пароля, визуальная чистота стартовых экранов.

css/dashboard.css

Стили кабинета: карточки метрик, карточки счетов, блоки быстрых действий, бюджеты, таблица последних транзакций.

css/transactions.css

Стили страницы транзакций: фильтры, таблица, карточки операций, блоки статистики и модальные окна редактирования.

css/reports.css

Стили аналитической страницы: карточки KPI, контейнеры графиков, визуальная иерархия дашборда отчётов.

css/integrations.css

Стили интеграций: карточки сервисов, таблица правил, история импортов и форма загрузки CSV.

css/responsive.css

Адаптивные правила для мобильных устройств и планшетов. интерфейс удобным на разных ширинах экрана без переписывания логики страниц.

Как связаны файлы между собой

Все внутренние страницы (dashboard, transactions, reports, integrations) подключают ui.js. Оттуда приходят:

- общая навигация (sidebar, topbar, footer) вставляется динамически, чтобы не дублировать разметку
- функции форматирования formatCurrency() и formatDate() единый стандарт для всего приложения
- showToast() для уведомлений чтобы сообщения об ошибках и успехах выглядели одинаково везде

Практически все JS-файлы используют storage.js как единственный источник правды. Никто больше не обращается к localStorage напрямую только через этот модуль.

- dashboard.js вызывает getDashboardMetrics() для получения сводки
- transactions.js использует filterTransactions() для списка операций
- reports.js работает с getReportData() для графиков
- integrations.js вызывает importTransactions() и applyRulesToTransaction()

dashboard.js, transactions.js, reports.js и integrations.js работают только после проверки авторизации. В начале каждого файла вызываю requireAuth() из storage.js. Если пользователь не залогинен, его сразу перекидывает на страницу входа. Так никто не сможет открыть личный кабинет или отчёты, просто введя URL вручную.

data.js подключается там, где нужны статические словари или демонстрационные данные:

- во время регистрации createDemoSeed() наполняет аккаунт пользователя стартовыми транзакциями и счетами
- на странице интеграций sampleImportCsv используется для тестового импорта без загрузки своего файла
- в формах списки категорий, валют и типов счетов подтягиваются из словарей data.js

Сохранение состояния:
Все изменения в данных сразу сохраняются в localStorage через storage.js.. Пользователь может закрыть вкладку, вернуться через час, и всё будет на месте:

добавила транзакцию → записалась в localStorage.изменила бюджет → обновилась запись.удалила счёт → исчез из хранилища